

P11 — Mini Evaluador de Expresiones Algebraicas

Fundamentos de Intérpretes y Evaluador de Expresiones

Integrantes del equipo:

Jose Luis Canchanya Ramos

Piero Mauro Morales Sinche

Aldair Edgard Gutierrez Mamani

Rodrigo Matias Ortiz Montoya

v2 — Mejorado

```
$ ./eval_math
> sin(pi/2)
= 1.000000
> x = 5 + e
= 7.718282
> funct
Funciones disponibles:
  sin(x) -> Seno
  ln(x)  -> Log natural
  ...
> list
```

Descripción del Proyecto

¿Qué es?

Evaluador interactivo de expresiones algebraicas escrito en C puro. Funciona como intérprete tipo REPL (Read-Eval-Print Loop) similar a GeoGebra. Construye un Árbol de Sintaxis Abstracta (AST) usando un Lexer y Parser de descenso recursivo, sin dependencias externas más allá de la biblioteca matemática estándar (-lm).

Herramientas

- ▶ GCC / Clang con -Wall -Wextra -lm
- ▶ GitHub con Makefile y CI automático
- ▶ Tests automáticos en cada push

6

Operadores

+ - * / ^ %

12+

Funciones

trig, log, rango

5

Módulos C

lexer, parser, eval...

Requerimientos Funcionales

RF1

Evaluación Aritmética

Suma +, Resta -, Multiplicación *, División /, Módulo %, Potencia ^

RF2

Soporte de Variables

Asignación y reutilización de variables ($a = x + y$) en toda la sesión

RF3

Funciones Matemáticas

exp, sqrt, pow | ln, log(a,b), log2, log10 | sin, cos, tan, asin, acos, atan, atan2 | max, min, abs

RF4

Manejo de Excepciones

Validación de división por cero y errores de dominio sin colgar la aplicación

RF5

Configuración Angular

Cambio de unidad entre Radianes y Grados mediante comandos rad / deg

RF6

Entorno REPL

Ciclo Read-Eval-Print-Loop con comandos list, funct, quit / exit

Pipeline: Lexer → Parser → Evaluador



Texto plano ingresado por el usuario en consola REPL

Tokeniza: números, identificadores, operadores. Descarta espacios.

Descenso recursivo según EBNF. Construye el AST con nodos tipados.

Recorre el AST en post-orden. Resuelve variables (symtable) y funciones.

Imprime resultado double o mensaje de error descriptivo.

Gramática EBNF — Precedencia (mayor → menor)

Paréntesis () > Unario -x +x > Potencia ^ (derecha→izq) > Mul / Div / Mod * / % > Suma / Resta + - > Asignación =

Ejemplo: `x = -2 ^ 3 + 4 * 5`

① -2 (unario)

② -2^3 = -8

③ 4*5 = 20

④ -8+20 = 12

⑤ x = 12

Modelo de Datos — Estructuras C

struct ASTNode

AST_NUMBER Valor flotante directo (hoja)

AST_VAR Nombre de variable → symtable

AST_BINOP Operador binario: left op right

AST_UNOP Operador unario: op left

AST_FUNC Llamada a función con args[]

AST_ASSIGN Asignación: nombre = right

```
union { double value; TokenType op; }
char name[32]; ASTNode *left, *right;
ASTNode **args; int num_args;
```

Tabla de Símbolos (symtable)

Arreglo estático de 100 entradas (MAX_VARIABLES). Cada Symbol almacena char name[32] + double value. Escritura inteligente: actualiza si existe, agrega si no. Pre-cargadas: pi, e, phi (constantes protegidas).

FuncEntry (callback table)

Arreglo global func_table[]. Sin cadenas if/else: cada entrada asocia el nombre con la función nativa de <math.h>. Para agregar una función nueva basta una sola línea en el arreglo.

```
char name[16]; int num_args;
MathFunc1 fn1; // f(x)
MathFunc2 fn2; // f(x,y)
```


NUEVO

Mejoras Implementadas — v2

Comando funct

Nuevo comando especial en el REPL que despliega todas las funciones disponibles, sus firmas, restricciones de dominio, y las constantes/operadores del evaluador.

$\ln(x)$ y $\log(a, b)$

Se renombró $\log(x) \rightarrow \ln(x)$ para evitar ambigüedad. Se agregó $\log(a, b)$: logaritmo en base a de b , con validaciones de dominio ($a > 0$, $a \neq 1$, $b > 0$).

Constantes protegidas

El texto del párrafo
 π , e y ϕ son ahora constantes de solo lectura. Se agregó ϕ (número áureo ≈ 1.618). Intentar reasignarlas genera un error descriptivo.

En desarrollo

Resolución de ecuaciones de grado 1–4, soporte para números imaginarios/complexos, y estadísticas sobre tablas de datos (media, mediana, moda, varianza, $\sigma \dots$).

NUEVO

Nuevo comando especial: "funct"

eval_math – bash

>> funct

Funciones matematicas disponibles:

sin(x) -> Seno

cos(x) -> Coseno

tan(x) -> Tangente

ln(x) -> Logaritmo natural ($x > 0$)

log(a, b) -> Log base a de b

log2(x) -> Log base 2

sqrt(x) -> Raiz cuadrada ($x \geq 0$)

abs(x) -> Valor absoluto

max(a,b)/min(a,b)/pow(a,b) ...

Constantes: pi e phi

Operadores: + - * / ^ %

Sin ambigüedad

El comando "funct" despliega la lista completa con firma y restricciones de dominio. El usuario sabe exactamente qué funciones existen y cómo usarlas.

Constantes visibles

Se listan pi, e y phi junto con sus valores. Aparecen marcadas como (constante) para indicar que no son modificables.

Operadores incluidos

La salida también enumera los 6 operadores disponibles con su símbolo para que el REPL sea completamente autodocumentado.

NUEVO

Cambios en Funciones Logarítmicas y Constantes

Funciones Logarítmicas Rediseñadas

ANTES

AHORA

`log(x)`

→

`ln(x)`

Logaritmo natural, sin ambigüedad

—

→

`log(a, b)`

Log base a de b. Valida: $a > 0$, $a \neq 1$, $b > 0$

`log2(x)`

→

`log2(x)`

Sin cambios (base 2)

`log10(x)`

→

`log10(x)`

Sin cambios (base 10)

Constantes Protegidas

pi

3.14159265358979...

Cociente perímetro/diámetro

e

2.71828182845904...

Base del logaritmo natural

phi

1.61803398874989...

Número áureo $\phi = (1+\sqrt{5})/2$ ♦ NUEVO

Intentar `pi = 3` genera:
Error: 'pi' es una constante protegida.

Funcionalidades en Progreso

01

Resolución de Ecuaciones

Grado 1, 2, 3, 4 y posiblemente superior

- Ecuaciones lineales: $ax + b = 0$
- Cuadráticas: fórmula general / discriminante
- Cúbicas y cuárticas: métodos de Cardano / Ferrari
- Interfaz: `solve(expr)` o comando dedicado

02

Números Imaginarios / Complejos

Ligado al soporte de ecuaciones

- Raíces complejas cuando $\Delta < 0$
- Representación $a + bi$
- Operaciones: suma, producto, módulo, argumento
- Constante i disponible como variable especial

03

Estadísticas sobre Tablas de Datos

Análisis de conjuntos numéricos ingresados

- Entrada: `data = {1, 2, 3, 4, 5}`
- Media, mediana, moda
- Varianza y desviación estándar
- Rango, mínimo, máximo, percentiles

Casos de Prueba

Entrada	Salida Esperada	Categoría
5 + 3 * 2	11	Precedencia
2 ^ 3 ^ 2	512	Asociatividad →
ln(e)	1	Función ln ♦ nuevo
log(10, 100)	2	log(a,b) ♦ nuevo
sin(pi/2)	1.000000	Trigonometría rad
sin(90) [modo deg]	1.000000	Modo grados
max(10-2, 5*2)	10	Funciones multi-arg
sqrt(pow(3,2)+pow(4,2))	5	Composición
phi * 2 - 1	2.23606...	Constante phi ♦ nuevo
log(-5)	Error dominio	Error esperado
pi = 3	Error: constante	Protección ♦ nuevo
5 / 0	Error div/0	Error esperado

¡Gracias!

P11 — Mini Evaluador de Expresiones Algebraicas

Entregable 1 — Análisis y Diseño | Fundamentos de Intérpretes

6 operadores

16+ funciones

3 constantes

funct cmd ✦

$\ln$ / $\log(a,b)$ ✦

Constantes prot. ✦

GitHub: github.com/felizmedioa/Proy_LP1_2026_1